Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Лабораторна робота № 2

з дисципліни «Системне програмування»

на тему «Інтеграція периферії, драйвер MPU6050 та перевірка BenchLevel»

Виконав:
студент групи ІО-35
Гордєєв Ілля Євгенович

Перевірив:
Гордієнко Нікіта Юрійович

Київ – 2026

Тема: Інтеграція периферії, драйвер MPU6050 та перевірка BenchLevel.

Мета роботи: Реалізувати драйвер MPU6050 та логіку індикатора нахилу, узгодити периферію STM32C031C6, перевірити цифрові транзакції й граничні умови за host-тестами та реальними записами Wokwi.

Постановка завдання

Побудувати прилад BenchLevel відповідно до PRD ЛР1. MPU6050 передає прискорення через I2C; потенціометр ADC задає допуск; кнопка GPIO калібрує еталон; UART і LED відображають результат. Вимоги FR-01..FR-13 включають калібрування, debounce, hysteresis, витримку переходу, помилки й відновлення.

Потрібно відокремити алгоритм від HAL, перевірити pinout, реалізувати неблокувальне опитування з обмеженням часу, зафіксувати нормальну й аномальну транзакції, вручну декодувати кадри та порівняти їх із драйвером і UART. Тести мають охоплювати драйвер, логіку й спільні події; звіт має містити результати, обмеження та виправлені проблеми.

Обрано форму B із Docker, STM32 HAL і Wokwi. Систему не переносили на інший SDK.

Хід роботи

Середовище та перевірені результати

Розроблено BenchLevel на STM32C031C6, Cortex-M0+, 48 МГц, 32 КіБ Flash, 12 КіБ RAM. Збережено STM32 HAL. Прошивку зібрано GCC ARM 13.2.1 у Docker Ubuntu 24.04.5 ARM64; host-тести виконано GCC 13.3.0 і gcovr 7.0. Wokwi VS Code 4.0.0 виконує HEX, PulseView 0.4.2 декодує записи.

Фактичні збірки, тести й захоплення виконано 16.10.2026. Дати Git є окремими метаданими та не датують вимірювання. Віддалений CI не запускався. Фізичну плату не випробувано. Додатково на VM Ubuntu 24.04.5 x86_64 виконано збірку ARM GCC 13.2.1, 19 host-тестів і coverage GCC/gcov 13.3. HEX збігається побайтно з Mac. Sigrok прочитав усі 10 SR-файлів; GUI Wokwi на VM не перевірявся.

Напрям	Артефакт	Результат
Периферія	firmware.ioc, pinout.md	Узгоджено I2C1, ADC1, USART2, GPIO та IRQ.
Архітектура	App, Devices, Platform	Логіку відокремлено від HAL контрактом Bus.
Драйвер	mpu6050.c	WHO_AM_I, конфігурація, 14 байтів, deadline.
Алгоритм	level.c, controller.c	Калібрування, допуск, LED, backoff.
Верифікація	tests, captures	19 host-тестів, Wokwi, VCD/SR, PulseView.
Документація	docs, PDF	PRD, джерела, вимірювання й висновки.

Усі наведені напрями виконав Гордєєв Ілля Євгенович. ЛР1 зафіксовано тегом lab1; зміни ЛР2 розміщено у feature/lab-2 з подальшим локальним злиттям у main. Коміт 145cbaf містить драйвер і логіку, 240fcf9 містить host-перевірки, a10b6c4 містить виправлення інтеграції та реальні захоплення.

Фінальна прошивка займає text=24816, data=104, bss=2680 байтів. Flash text+data становить 24920 із 32768 байтів. Залишок RAM без урахування стеку становить 9504 байти. Піковий стек не вимірювався.

Розведення та електричні обмеження

Функції виводів перевірено за DS13867 Rev 4, таблиці 12–15, с. 31–36; роз'єми за UM2953 Rev 2, таблиця 11, с. 20. RM0490 задає режими GPIO, ADC, I2C та USART.

Сигнал	МК / роз'єм	Підключення та режим
SCL	PB8, AF6 / D15, CN5.10	I2C1 open-drain, MPU SCL, LA D0.
SDA	PB9, AF6 / D14, CN5.9	I2C1 open-drain, MPU SDA, LA D1.
UART TX/RX	PA2/PA3, AF1 / Morpho	USART2 115200 8N1; TX до monitor RX, LA D2.
Потенціометр	PA0 / A0, CN8.1	ADC1_IN0, analog, 12 bit, SIG 0..3,3 В.
CAL	PB0 / D10, CN5.3	Input pull-up, кнопка на GND, LA D3.
LED	PA5 / D13, CN5.6	Бортовий LD4, output, LA D4.
ADC marker	PA4 / A2, CN8.3	Фронт на кожне завершення ADC, LA D5.
Bus marker	PA6 / D12, CN5.5	Фронт на початок операції Bus, LA D6.

MPU6050 живиться від 3V3, AD0 на GND задає 0x68. SCL і SDA мають pull-up 4,7 кОм до 3V3. Потенціометр 10 кОм підключено між 3V3 та GND; усі компоненти мають спільну землю. Режим ADC fixed sequencer узгоджений у коді та firmware.ioc.

PA2/PA3 USART2 не є Arduino D1/D0, якими тут є PB6/PB7 USART1. Альтернатива I2C1 PB6/PB7 AF6 можлива за datasheet, але не обрана, щоб уникнути конфлікту з Arduino UART. PC13 не використано як CAL.

MPU6050 споживає типово 3,8 мА з accel+gyro та вимкненим DMP; потенціометр 0,33 мА, два pull-up до 1,40 мА разом, кнопка менше 0,2 мА. Сума близько 5,73 мА, це не гарантований піковий струм модуля. Проєктний резерв 20 мА не вимірювався. UM2953, §6.5.2, с. 14 обмежує сумарний струм плати та навантаження 3V3 до 500 мА. Власне споживання Nucleo треба враховувати окремо.

Ідеальні фронти Wokwi не підтверджують фізичний rise time, запас живлення або точність ADC. На breakout треба перевіряти фактичні pull-up та допустиме живлення.

Розрахунок шин і правила інтеграції

HSI=48 МГц, дільники SYSCLK, AHB та APB дорівнюють 1. Для I2C1 kernel=48 МГц. TIMINGR=0x20303E5D задає PRESC=2, тому $t_{PRESC} = 3/48$ МГц $= 62{,}5$ нс. SCLL=93 дає 5,875 мкс, SCLH=62 дає 3,9375 мкс, SCLDEL=3 дає 250 нс. Сума low/high 9,8125 мкс не включає синхронізацію, фільтри та фізичні фронти. Ціль близько 100 кГц узгоджується з Wokwi 5/5 мкс, але модель не доводить фізичні затримки.

Для USART2 prescaler=1, oversampling=16, BRR приблизно $\text{round}(48000000/115200) = 417$. Оцінка baud=115107,9, відхилення близько $-0{,}08\%$. ADC PCLK/4=12 МГц, sampling 79,5 та conversion 12,5 циклів, номінально близько 7,67 мкс. Програмний інтервал 50 мс більший за conversion.

Відхилено I2C1 PB6/PB7 AF6, щоб лишити Arduino USART1 D1/D0 вільними. Варіант USART1 PB6/PB7 AF0 потребує перенесення початкового UART/VCP тракту; збережено USART2 PA2/PA3. SWD PA13/PA14 вільні; DMA-запитів і PWM немає. SysTick дає часову базу, I2C/USART мають окремі IRQ.

HAL ADC потрібен потенціометру, I2C датчику, UART телеметрії, GPIO кнопці й LED, RCC/PWR/FLASH clock і latency, CORTEX SysTick/NVIC. DMA залишено як залежність типів і helper API HAL, канали не налаштовані. Невикористаний EXTI вимкнено та виключено зі збірки. HEX залишився ідентичним захопленій версії.

App/Devices/Platform розташовані поза згенерованими каталогами. Виклики у main збережено в USER CODE. ADC init/MSP підтримуються вручну разом з IOC. Повторну генерацію CubeMX не виконували; вона потребує порівняння diff, збереження IRQ у Platform і повторної перевірки контрактів.

Компоненти програмного забезпечення

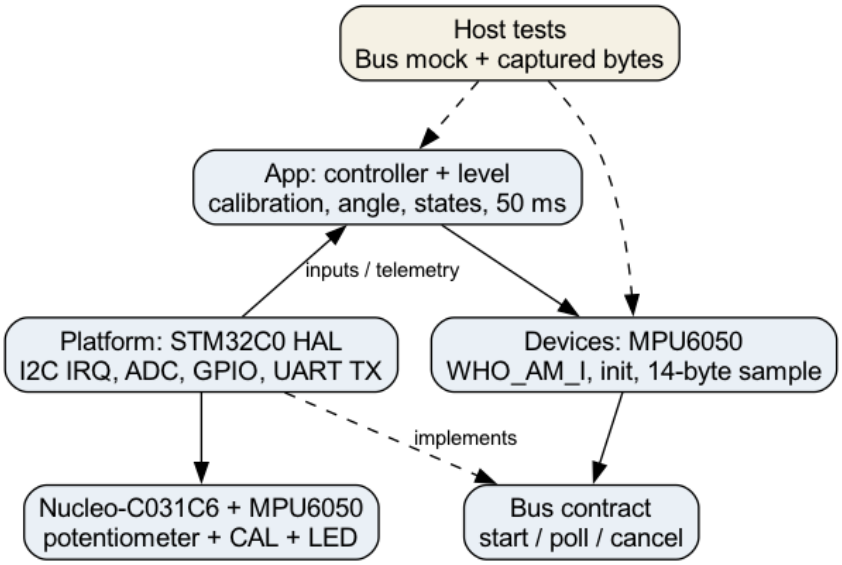


Рисунок 1 – Залежності компонентів і межа host-тестів

App працює лише з Devices API та стандартною C. Devices знає контракт Bus, але не включає HAL. Platform реалізує Bus через HAL та обслуговує ADC, GPIO і UART. Core налаштовує clock, MSP, IRQ та викликає Platform.

Bus має start, poll, cancel та результати PENDING, OK, NACK, TIMEOUT, ERROR, BUSY. Драйвер передає 7-бітну адресу; Platform зсуває її один раз для HAL. Дані й буфери живуть до завершення операції. Deadline 10 мс обмежує як PENDING, так і BUSY.

У драйвері послідовно виконуються WHO_AM_I 0x75, перевірка 0x68, PWR_MGMT_1 = 1, очікування 100 мс, CONFIG 0x1A=3, GYRO_CONFIG 0x1B=0, ACCEL_CONFIG 0x1C=0. Далі 14 байтів від 0x3B перетворюються з signed big-endian на accel, temperature, gyro.

Platform передає номер регістра через IRQ, завершує STOP і окремо запускає IRQ-приймання. Саме цей шлях підтверджено Wokwi. Режим HAL Sequential з repeated START у моделі давав SIZE=64 і timeout. Працездатність repeated START не заявлено. Власний IRQ-код вимикає TXIE/RXIE при XferCount=0, залишаючи STOP/NACK/error. Vendor HAL не змінено.

Калібрування, стани та неблокувальний цикл

ADC запускається кожні 50 мс, EOC перевіряється без очікування; deadline 5 мс захищає від зависання. Main не викликає HAL_Delay або printf. UART передає рядок через interrupt TX із постійного буфера; зайнятий передавач не перезаписується.

Коротке натискання CAL після debounce 30 мс починає накопичення 20 послідовних відліків. Приймається модуль 0,85..1,15 g. Зміна нормованого вектора понад 0,02 між сусідніми відліками скидає накопичення. Середній нормований вектор стає еталоном у RAM.

Поточний вектор фільтрується з $\alpha = 0{,}25$. Кут $\theta = \arccos(\hat{g}_{ref} \cdot \hat{g}_{filtered})$. Добуток обмежується до $[-1; 1]$, майже нульовий вектор після фільтра запускається заново, щоб уникнути ділення на нуль. Допуск у міліградусах:

$$T = 500 + \left\lfloor \frac{ADC \cdot 4500}{4095} \right\rfloor.$$

При $\theta > T$ кандидат TILT має зберігатися 300 мс. Повернення LEVEL потребує $\theta + 200 \leq T$ та такої самої витримки.

Стан	Умова	LED
UNCAL	Немає еталона	Погашений.
CAL	Накопичення еталона	Перемикання кожні 100 мс.
LEVEL	Кут у допуску	Постійно ввімкнений.
TILT	Кут поза допуском	Перемикання кожні 500 мс.
ERROR	Bus/ADC/stale	Перемикання кожні 125 мс.

Відсутність придатного відліку понад 250 мс або помилка Bus скидає калібрування й переводить у ERROR. Повторний init 0x68 відбувається через 1000 мс. Після успіху стан UNCAL, потрібне нове натискання CAL.

Довге утримання CAL 2 с ставить один діагностичний запит 0x69 до відпускання. Це перевіряє NACK при незмінній схемі. Введення команд UART у поточному інтерфейсі не використовується. Контракт i/n залишено для host-тестів контролера.

Інтеграційна перевірка Wokwi

Після запуску є службова затримка 500 мс для приєднання аналізатора. Рядок t=500 selftest=WAIT з ERROR до готовності ADC та IMU є перехідним станом. Близько t=607 мс перевірено selftest=PASS, id=68, adc=1365, tol_mdeg=2000, ax=0, ay=0, az=16384, norm_mg=1000 та UNCAL.

При Acceleration X=0, Y=0, Z=1 g і короткому CAL система переходить CAL, потім LEVEL. На фінальному короткому записі потенціометр встановлено в мінімум, UART показує ADC=0, T=500 mdeg та angle=0.

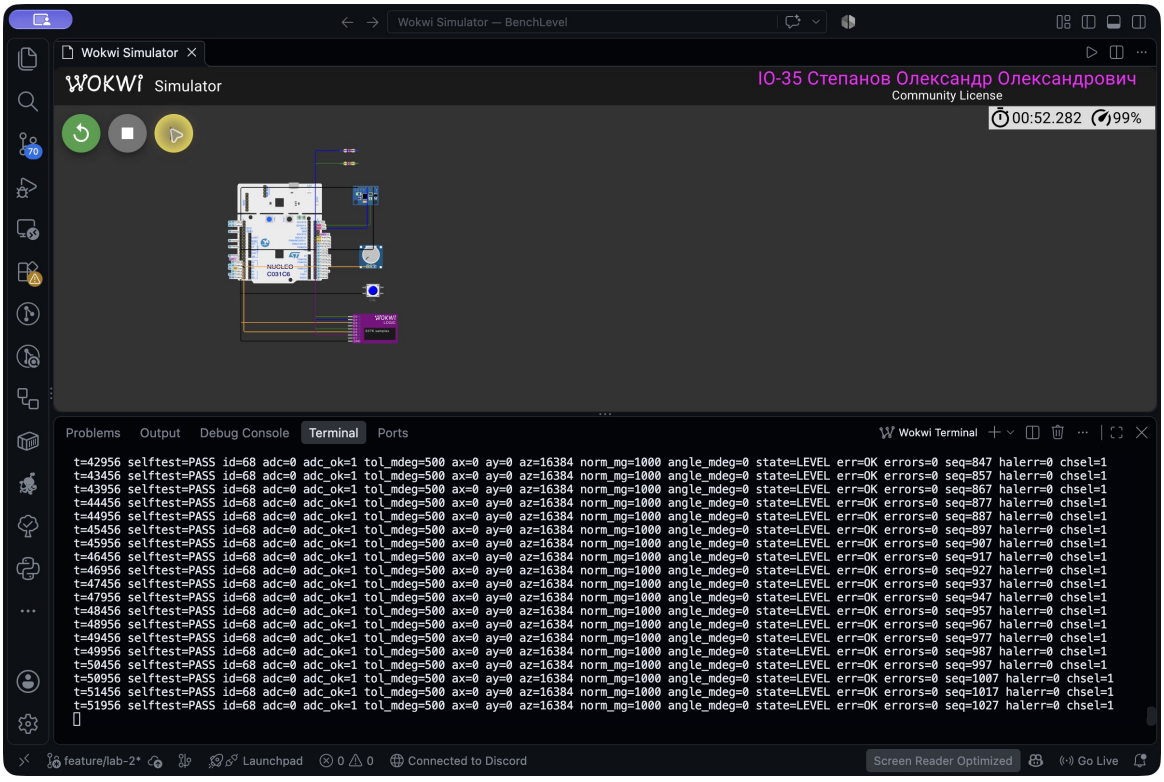


Рисунок 2 – Wokwi: відкалібрована площина, LEVEL, ADC=0

Скріншоти є справжніми знімками Wokwi; текстові копії доступного UART viewport збережено у verification/lab2. Viewport є фрагментом потоку, тому повний обмін перевірено окремим UART decoder із запису.

Збірка та схема використовують Nucleo-C031C6 і HAL STM32C0. Параметри Cortex-M4 або FPU з інших прикладів до цієї плати непридатні. Фізична точність вимірювання кута не перевірялася.

Нахил і регулювання допуску

Після калібрування при X=0, Y=0, Z=1 g встановлено Acceleration X=0,6, Y=0, Z=0,8 g. Норма залишається близькою до 1 g, теоретичний кут до осі Z дорівнює arccos(0,8) ≈ 36,87˚. UART показує ax=9830, az=13107, angle_mdeg=36869 та TILT.

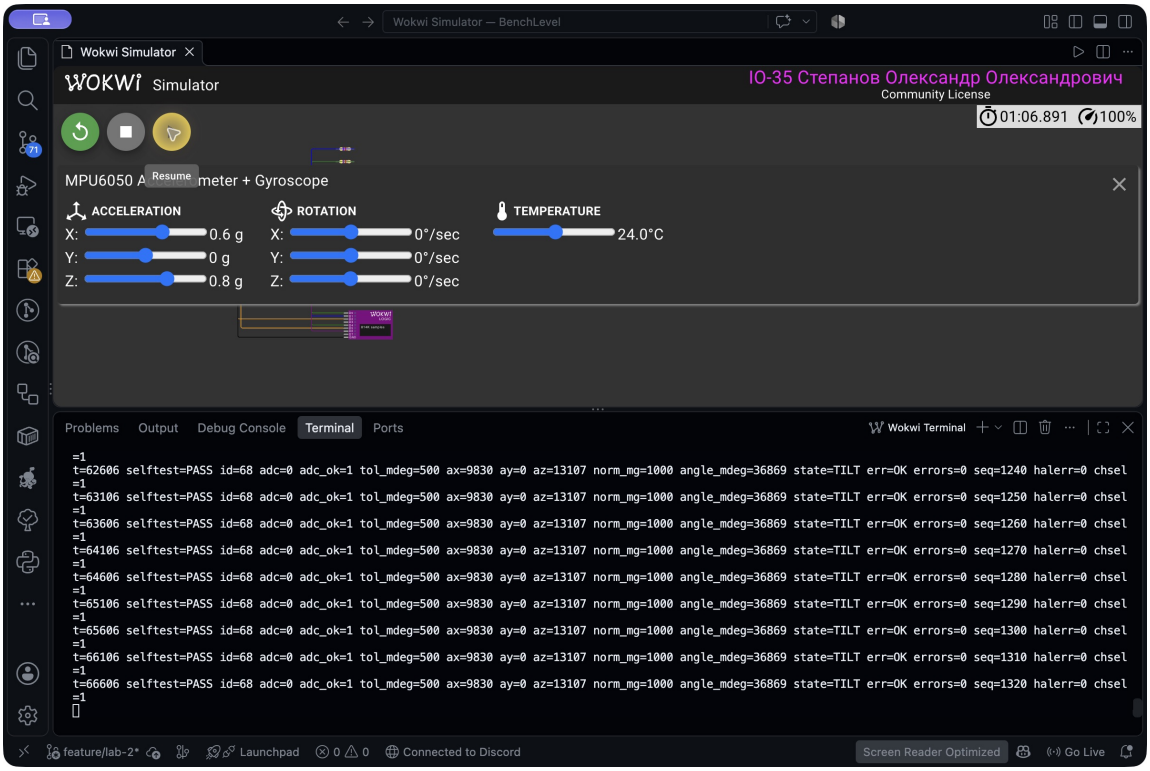


Рисунок 3 – Wokwi: заданий нахил 0,6/0/0,8 g і стан TILT

У захопленні tilt.vcd ці самі accel-байти мають значення 0x2666 і 0x3333. Тест captured_bus_vector передає їх у той самий драйвер, який виконується в прошивці, і порівнює норму та кут із UART.

Home та End на потенціометрі дають ADC=0 і ADC=4095, відповідно T=500 та T=5000 mdeg. Це підтверджено окремими знімками pot-min.png і pot-max.png. Поля Rotation MPU6050 задають кутову швидкість, тому нахил у цій перевірці змінено через Acceleration.

Алгоритм визначає відносний нахил за гравітацією, але не yaw. Лінійне прискорення та вібрація можуть спотворити результат. Відкидання норми та калібрування не замінюють метрологічну перевірку.

I2C: нормальне читання і ручне декодування

Логічний аналізатор підключено D0=SCL, D1=SDA; raw VCD має крок 1 нс. SR отримано через libsigrok з downsample=100, тобто 10 МГц. Це близько 100 відліків на період SCL 100 кГц. Крок VCD є цифровою сіткою моделі. Trigger вимкнений, запис безперервний від старту симуляції, вікна обрано після захоплення. Спільний GND підключено.

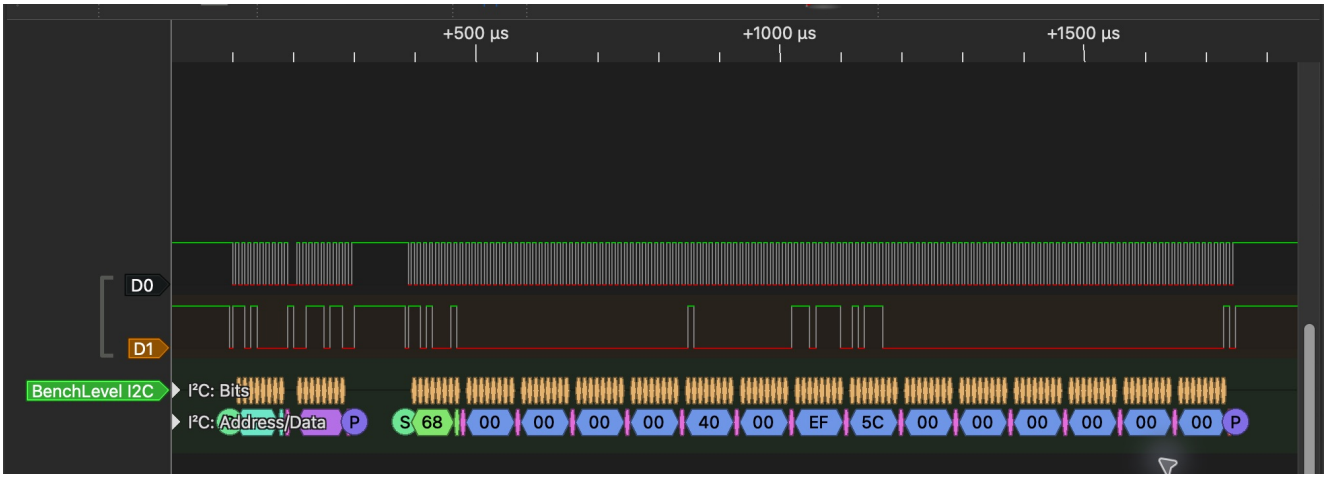


Рисунок 4 – PulseView: адреса 0x68, read і 14 байтів MPU6050

На фронтах SCL біти адреси дорівнюють 11010001, тобто 0xD1: сім біт адреси 0x68 і read=1. Дев'ятий біт 0 означає ACK від slave. Після передачі покажчика 0x3B із STOP окреме читання повертає:

```
00 00 00 00 40 00 EF 5C 00 00 00 00 00 00
```

Ax=0, Ay=0, Az=0x4000=16384; при 16384 LSB/g це 1 g. Температура 0xEF5C=-4260, приблизно $-4260/340 + 36{,}53 = 24{,}00^{\circ}C$. Gyro XYZ=0. UART містить ті самі значення.

Останній дев'ятий біт 1 після даних є нормальним master NACK перед STOP. Він завершує читання і не є помилкою slave. При SCL high/low по 5 мкс read займає 1365 мкс. Розрахунок $15 \cdot 9 \cdot 10 = 1350$ мкс плюс 15 мкс на START/STOP збігається з записом 606683250..608048250 нс.

Хеш джерела, початкові рівні та зсув вікна збережені. tools/analyze_capture.py незалежно витягає біти й формує captured_vector.h. Між STOP pointer і read START 83,275 мкс. Фронт D6 перед викликом HAL випереджає pointer START на 19,525 мкс.

I2C: адресний NACK і відновлення

Утримання CAL запускає write до 0x69 при AD0=GND і незмінній адресі датчика 0x68. SDA дає 11010010, тобто 0xD2: адреса 0x69, write=0. Дев’ятий біт 1 означає address NACK.

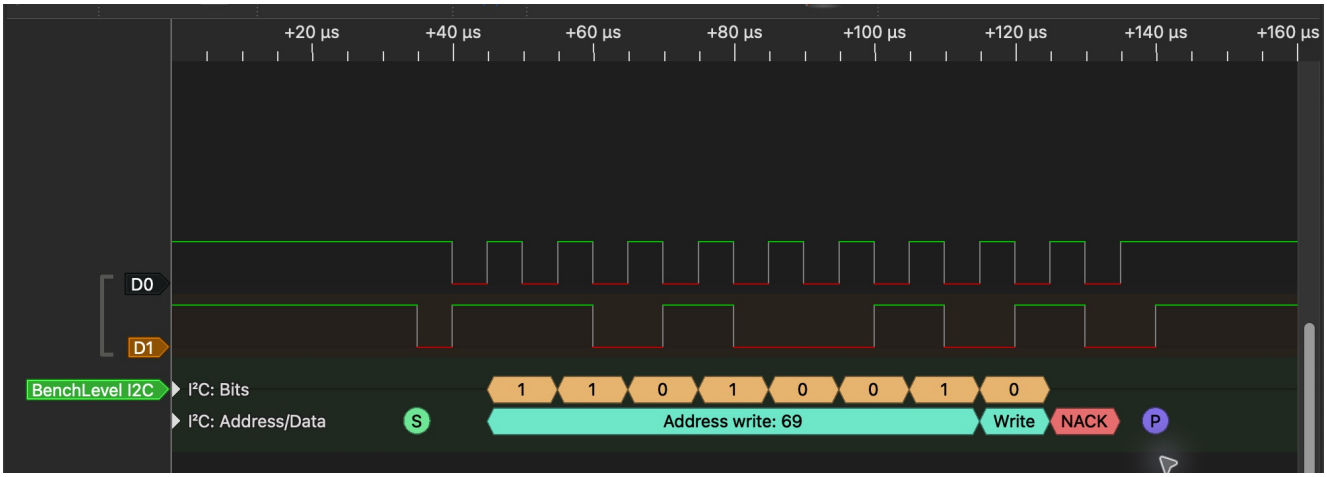


Рисунок 5 – PulseView: 0x69 write, address NACK і STOP

Аномальна транзакція триває 105 мкс, 24527384825..24527489825 нс первинного запису. Вона не містить даних датчика. UART показує selftest=FAIL, err=NACK, halerr=4, errors=1 і ERROR. Це керована перевірка відсутньої адреси, фізичний обрив дроту не моделювався.

Контролер очікує 1000 мс перед повторним init 0x68. Окремий справжній знімок recovered-uncal.png підтверджує PASS, id=68, errors=1 та UNCAL після відновлення. Цей пізніший UI-момент не входить у 25-секундний address-nack.vcd; зв’язок між файлом і знімком не підмінено.

Тести окремо перевіряють NACK, timeout, BUSY, неправильний ID, помилку старту, backoff і переповнення uint32 часу. Після відновлення еталон скинуто, тому прилад не показує LEVEL без повторного калібрування.

UART та період ADC

UART декодовано як 115200, 8 data bits, no parity, 1 stop bit, LSB first. Номінальний біт 8,6806 мкс; короткі інтервали фронтів 8,675/8,700 мкс узгоджуються з моделлю та обслуговуванням IRQ.

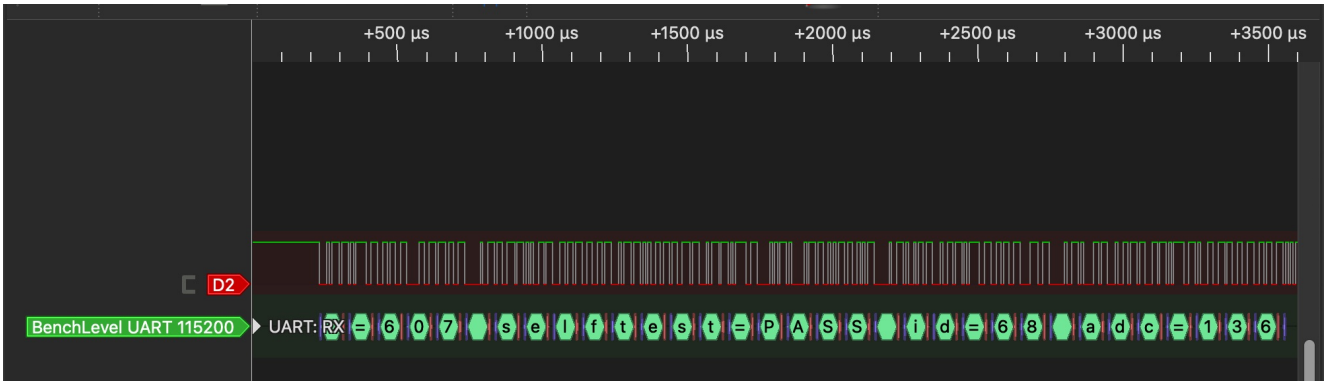


Рисунок 6 – PulseView: ASCII t=607 selftest=PASS id=68 adc=1365

D5 перемикається на кожне завершення ADC. Один фронт, а не повний період прямокутника, відповідає одному вимірюванню.

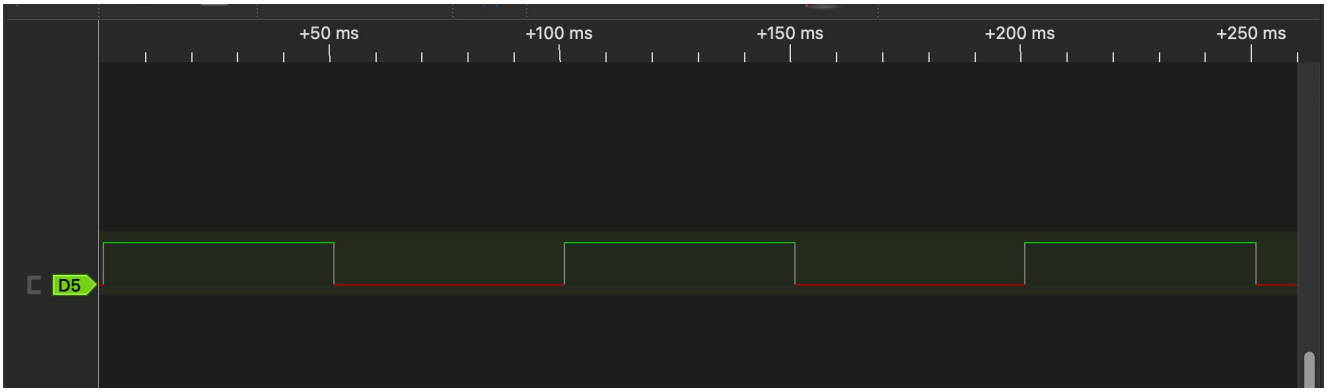


Рисунок 7 – PulseView: фронти маркера ADC через приблизно 50 мс

Для ділянки після 600 мс min/median/max інтервалу становлять 49,918225 / 49,971475 / 50,086700 мс. Повний період D5 близько 100 мс. Стартовий інтервал до першого ADC виключено з оцінки періоду.

GPIO та автоматизовані перевірки

D3 є active-low CAL, D4 є LED. Після натискання і debounce спостерігається CAL із перемиканням LED кожні 100 мс.

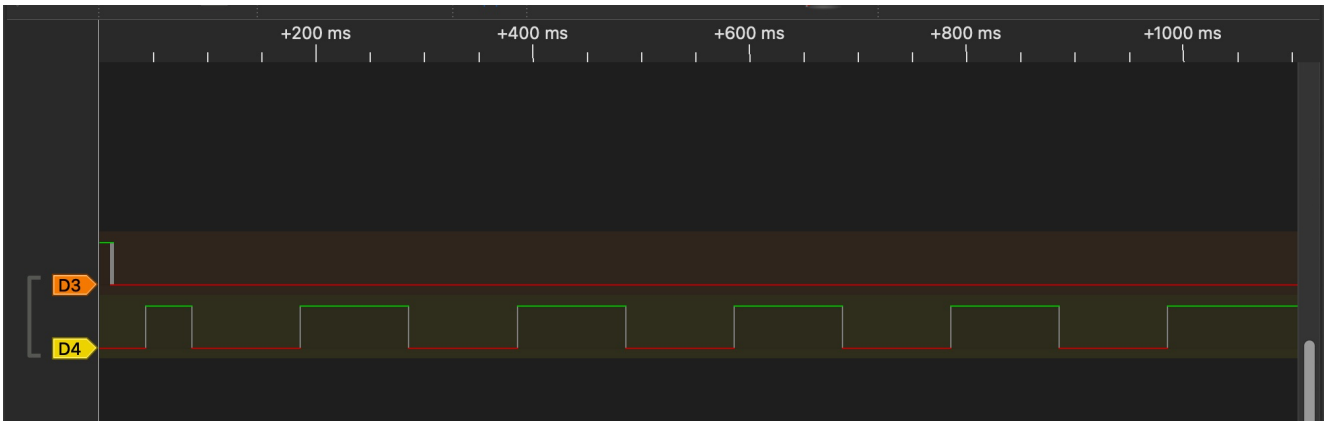


Рисунок 8 – PulseView: натиснутий CAL і індикація калібрування

Виконано `make clean`, `make build`, `make test`, `make coverage`. Усі 19 тестів пройшли; фінальний лог перевірок містить фактичний час UTC.

Використано невеликий C assert-runner і mock Bus, який перевіряє порядок, адресу, регістр, довжину, дані та результати poll/cancel. Тестування завершується на першому assert, окремої бібліотеки mocking немає.

Група	Перевірені випадки
Драйвер, 8 тестів	Порядок init, settle, NACK/IO, start faults, ID, timeout wrap, BUSY, signed units.
Логіка, 8 тестів	ADC endpoints, bounce/hold, motion/invalid CAL, angle/dwell, hysteresis/cancel, stale/recover/wrap, LED, opposite filter.
Інтеграція, 3 тести	Одночасні події, один long press, captured flat і tilt bytes.

App/Devices мають 190/192 виконаних рядків, 99,0%; 148/157 гілок, 94,3%; 16/16 функцій. Два непокриті рядки драйвера є захисними гілками недосяжної внутрішньої фази/default. HAL і Platform виключено з host-coverage, їх інтеграцію перевірено Wokwi та захопленнями. Скрипт перевірки контрактів звіряє include allowlists, IOC, board.h і diagram.json. Gate 90% рядків обрано для невеликих чистих модулів; фактичне покриття його перевищує, захисні гілки лишаються видимими.

Проблеми та виправлення

Host-тести не виявили окремого дефекту алгоритму у виконаних випадках. Інтеграційні проблеми нижче отримано зі справжніх помилок збірки, моделі та інструментів.

Спостереження	Причина / перевірка	Виправлення та коміт
Host GCC не знаходив stdint.h	У Docker був gcc без libc6-dev.	Додано libc6-dev; host-тести збираються. 240fcf9.
I2C pointer ACK, далі timeout і HAL SIZE=64	Sequential HAL у Wokwi не завершував очікуваний read; перший VCD збережено.	Окремі TX/STOP/RX, deferred receive, guard IRQ. a10b6c4.
ADC змінювався при сталому потенціометрі	Невідповідний sequencer у моделі; chsel і код не узгоджувалися.	Fixed single-channel sequencer; endpoints 0/4095 перевірено. a10b6c4.
gcovr показував 0 рядків	Відносний filter не збігався з шляхами ../firmware.	Виправлено filter; coverage 190/192. a10b6c4.
VCD закінчився на 79,605 с	Буфер 1 млн змін заповнився; пізні UI-події відсутні.	Буфер 4 млн, окремий короткий tilt-source; старий запис позначено. a10b6c4.
UART input не підтверджено	Надсилання з monitor не дало перевіреного керування.	Діагностику перенесено на long CAL; вхід UART не заявлено. a10b6c4.

Первинні невдалі записи, знімки та логи залишено поруч із успішними; вони мають явні назви first-failure та first-simulator-failure. Первинний address-nack.vcd має SHA-256 `3e0f4a36...082f2cff`; повний хеш є у measurements.json. Для tilt-source збережено хеш розпакованого VCD `e4ae2d54...1ee5f04c4`.

На VM початковий GCC 9.5 не відповідав системному gcov 13.3, тому coverage завершувався signal 11. Вибір наявних gcc-13 і gcov-13 без зміни системних alternatives дав ті самі 190/192 рядків. Обидва логи збережено.

Dockerfile використовує ubuntu:24.04 та apt без фіксації версій. Дві чисті збірки ЛР1 в одному локальному образі дали однакові SHA-256. Майбутній apt update або інша архітектура host можуть змінити байти; версії та vendor-хеші зафіксовані в docs/versions.md і vendor-sha256.json.

Висновки

Реалізовано BenchLevel із MPU6050, ADC та CAL. Перевірено WHO_AM_I=0x68, читання 14 байтів, калібрування площини, допуск 0,5..5°, перехід LEVEL/TILT, адресний NACK і відновлення до UNCAL. При прискоренні 0,6/0/0,8 g драйвер отримує 9830/0/13107, а алгоритм показує 36,869°, що узгоджується з геометрією моделі.

Ручне декодування підтверджує 0x68 read, ACK і дані при SCL 100 кГц. Master NACK після останнього байта є нормальним завершенням read; address NACK для 0x69 спричиняє ERROR. UART decoder незалежно підтверджує selftest=PASS. Маркер ADC показує інтервали близько 50 мс.

Усі 19 host-тестів пройшли, coverage App/Devices становить 99,0% рядків. Це не доказ відсутності всіх дефектів. Фізичні електричні характеристики, точність кута, шум, температурний дрейф та піковий стек не виміряно. Окреме обмеження інтеграції: Wokwi перевірив STOP між pointer write і read, а не repeated START. Еталон зберігається лише в RAM.

Первинні джерела

1. ST DS13867 Rev 4. Виводи й AF, таблиці 12–15, с. 31–36; ADC characteristics.
2. ST UM2953 Rev 2. §6.5.2, с. 14, живлення; таблиця 11, с. 20, роз’єми.
3. ST RM0490. Розділи GPIO, ADC, I2C, USART.
4. TDK MPU-6000/6050 Product Specification Rev 3.4. §6, с. 13–14; живлення й типові струми.
5. TDK Register Map Rev 4.2. Регістри 26–28, 59–72, 107 і 117.
6. Wokwi MPU6050, potentiometer, pushbutton, logic analyzer. Поля Acceleration, межі потенціометра, bounce, bufferSize.
7. PulseView 0.4.2 manual. Декодери, сесії та автозавантаження PVS.
8. ST STM32C0 HAL v1.0.1. Джерело доданих ADC-модулів; наявний HAL збережено.